22.04.2019

EE 241 DIGITAL CIRCUITS
MIDTERM 2 SOLUTION MANUAL

Name:

Q1) (60 points) Enigma is an electro-mechanical ciphering machine used extensively during WW-II. Unlike Cesar's cipher, here the amount of shift in the letter sequence depends on both 8-bit ASCII value of the letter and amount of shift in the previous cycle. Design an Enigma machine with the following specifications and write down its Verilog description as follows.

a) (10 points) You have an 8-bit input for representing the ASCII-code of a letter, and 8-bit output for representing the ASCII-code of encrypted letter. Needless to say, you have a clock input as all sequential circuits require. Define all these values in this part.

Solution:

```
input [7:0] in;
input clk;
output reg [7:0] out;
```

b) (10 points) There are four states each of which shifts the 8-bit input by adding or subtracting certain amount of value to the ASCII value of the input. The states with the numbers and functionalities are listed as follows:
State #1: Add-by "3"
State #2: Add-by "8"
State #3: Subtract-by "2"
State #4: Subtract-by "6"

Define all these states in this part.

Solution:

```
parameter s1=2'b00,s2=2'b01,s3=2'b10,s4=2'b11;
reg [1:0] pstate,nstate;
```

c) (30 points) Transition between states is performed by checking the total number of 1s in the ASCII character. After shifting (adding or subtracting depending on the state), the state number will increase if the total number of 1s in the new ASCII character is even. Likewise, the state number will decrease if the total number of 1s in the new ASCII character is odd. If the state number equals to #4, increasing the state number corresponds to transition to state #1.

Likewise, if the state number equals to #1, decreasing the state number corresponds to transition to state #4.

Define all these transitions in this part.

Solution:

```
output parity;
assign parity = ~(^in);

initial
{out,nstate} = {8'd0,s1};

always @(posedge clk)
begin
pstate = nstate;
case (pstate)
   s1: if (parity) {nstate,out} = {s2,in+8'd8};
      else       {nstate,out} = {s4,in-8'd6};
   s2: if (parity) {nstate,out} = {s3,in-8'd2};
       else       {nstate,out} = {s1,in+8'd3};
   s3: if (parity) {nstate,out} = {s4,in-8'd6};
      else       {nstate,out} = {s2,in+8'd8};
   s4: if (parity) {nstate,out} = {s1,in+8'd3};
       else       {nstate,out} = {s3,in-8'd2};
endcase
end
```

d) (10 points) Write down the complete Verilog description based on the above parts.

Solution:

```
module main(clk,parity,in,out);
input [7:0] in;
input clk;
output reg [7:0] out;

parameter
s1=2'b00,s2=2'b01,s3=2'b10,s4=2'b11;
reg [1:0] pstate,nstate;

output parity;
assign parity = ~(^in);

initial
{out,nstate} = {8'd0,s1};
```

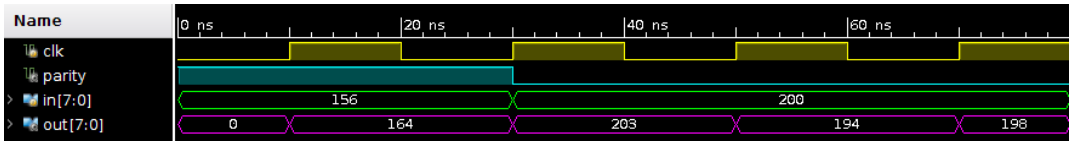
```
module testbench;
reg clk;
reg [7:0] in;
wire [7:0] out;
wire parity;

main u1(clk,parity,in,out);

initial begin
clk=1'b0;
repeat (8)
#10 clk=~clk;
$stop;
end
```

```
always @(posedge clk)
begin
pstate = nstate;
case (pstate)
   s1: if (parity) {nstate,out} = {s2,in+8'd8};
      else       {nstate,out} = {s4,in-8'd6};
   s2: if (parity) {nstate,out} = {s3,in-8'd2};
       else       {nstate,out} = {s1,in+8'd3};
   s3: if (parity) {nstate,out} = {s4,in-8'd6};
      else       {nstate,out} = {s2,in+8'd8};
   s4: if (parity) {nstate,out} = {s1,in+8'd3};
       else       {nstate,out} = {s3,in-8'd2};
endcase
end
endmodule
```

```
initial begin
in=8'd156; #30;
in=8'd200; #30;
end
endmodule
```



Q2) (40 points) Form the state diagram of the sequential circuit with the following Verilog description.

```
module main(clk,in,out);
parameter A=2'b00, B=2'b01, C=2'b10, D=2'b11;
input in, clk;
output reg [2:0] out;
reg [1:0] pstate, nstate;

initial
{out,nstate}={3'b100,A};

always @(posedge clk)
   begin
   pstate = nstate;
   case (pstate)
      A: if (in)
         {nstate,out} = {B,3'b000};
         else
         {nstate,out} = {D,3'b111};
      B: if (in)
         {nstate,out} = {C,3'b001};
          else
         {nstate,out} = {A,3'b100};
      C: if (in)
         {nstate,out} = {D,3'b010};
         else
         {nstate,out} = {B,3'b101};
      D: if (in)
         {nstate,out} = {A,3'b011};
          else
         {nstate,out} = {C,3'b010};
   endcase
   end
endmodule
```

Solution:

